

LTspice_Circuit_Wrappers

```
import os
import numpy as np
from PyLTSpice import RawRead
import re
import glob
from PyLTSpice import SimCommander
import subprocess
from SpiceFunctions import SpiceEvaluator , SpiceWaveformReader


def evaluate_miller_ota_ltspice(Wd, Ld, Wm, Lm, Wss, Lss, WB, Ib, CL,
                                Vicm,
                                beta, Rz, Cc,
                                TGT_VICM_MIN, TGT_VICM_MAX, TGT_VOUT_MIN,
                                TGT_VOUT_MAX,
                                BASE_DIR=".", **kwargs):

    LTSPICE_PATH =
r"C:\Users\Mohamed\AppData\Local\Programs\ADI\LTspice\LTspice.exe"
    """
    Executes NGSpice/LTspice testbenches for a Miller OTA and returns a
    specs dictionary.
    """

    # 1. Define Parameter Dictionaries
    parameters = {
        'Ld': Ld,
        'Wd': Wd,
        'Lm': Lm,
        'Wm': Wm,
        'LSS': Lss,
        'WSS': Wss,
        'WB': WB,
        'beta': beta,
```

```
    'Rz': Rz,  
    'Cc': Cc,  
    'Ib': Ib,  
    'vicm': Vicm,  
    'CL': CL  
}
```

```
parameters.update(kwargs)
```

```
CMIR_min_params = parameters.copy()  
CMIR_min_params['vicm'] = TGT_VICM_MIN
```

```
CMIR_max_params = parameters.copy()  
CMIR_max_params['vicm'] = TGT_VICM_MAX
```

```
VOUT_min_params = parameters.copy()  
VOUT_min_params['vicm'] = TGT_VOUT_MIN
```

```
VOUT_max_params = parameters.copy()  
VOUT_max_params['vicm'] = TGT_VOUT_MAX
```

2. Setup File Paths

```
full_path1_AC = os.path.join(BASE_DIR, "Miller_P_AC.sp")  
full_path1_DC = os.path.join(BASE_DIR, "Miller_P_DC.sp")  
full_path1_BUFFER = os.path.join(BASE_DIR, "Miller_P_buffer.sp")
```

```
# Define absolute path for the generated raw file
```

```
raw_path_ac = os.path.join(BASE_DIR, "Miller_P_AC.raw")
```

3. Initialize Evaluators

```
Miller_evaluator_ac = SpiceEvaluator(full_path1_AC,  
ltspice_path=LTSPICE_PATH)  
Miller_evaluator_dc = SpiceEvaluator(full_path1_DC,  
ltspice_path=LTSPICE_PATH)  
Miller_evaluator_CMIR_min = SpiceEvaluator(full_path1_AC,  
ltspice_path=LTSPICE_PATH)  
Miller_evaluator_CMIR_max = SpiceEvaluator(full_path1_AC,  
ltspice_path=LTSPICE_PATH)  
Miller_evaluator_VOUT_min_err = SpiceEvaluator(full_path1_BUFFER,
```

```

ltspice_path=LTSPICE_PATH)
    Miller_evaluator_VOUT_max_err = SpiceEvaluator(full_path1_BUFFER,
ltspice_path=LTSPICE_PATH)

#
=====

# 4. RUN AC SIMULATIONS
#
=====

metrics_ac = Miller_evaluator_ac(**parameters)

# FIX: Wrapped all waveform reader paths with absolute paths inside
BASE_DIR
reader1 = SpiceWaveformReader(raw_path_ac)
freq, gain_db, phase_deg = reader1.get_ac_response("V(vout_dm)")

ugf_idx = np.argmin(np.abs(gain_db))
UGF = freq[ugf_idx]
PM = 180.0 - np.abs(phase_deg[ugf_idx] - phase_deg[0])

metrics_ac['UGF'] = float(np.real(UGF))
metrics_ac['AVD'] = float(np.max(gain_db))

# Get Common-Mode AC response
reader2 = SpiceWaveformReader(raw_path_ac)
freq_cm, gain_db_cm, phase_deg_cm =
reader2.get_ac_response("V(vout_cm)")
metrics_ac['AVCM'] = float(gain_db_cm[0])
metrics_ac['CMRR'] = metrics_ac['AVD'] - metrics_ac['AVCM']

# CMIR Extremes
metrics_CMIR_min = Miller_evaluator_CMIR_min(**CMIR_min_params)
reader3 = SpiceWaveformReader(raw_path_ac)
_, gain_db_min, _ = reader3.get_ac_response("V(vout_dm)")
metrics_CMIR_min['AVD'] = float(np.max(gain_db_min))

metrics_CMIR_max = Miller_evaluator_CMIR_max(**CMIR_max_params)
reader4 = SpiceWaveformReader(raw_path_ac)
_, gain_db_max, _ = reader4.get_ac_response("V(vout_dm)")

```

```
metrics_CMIR_max['AVD'] = float(np.max(gain_db_max))
```

```
#
```

```
=====
```

```
# 5. RUN DC SIMULATION
```

```
#
```

```
=====
```

```
metrics_dc = Miller_evaluator_dc(**parameters)
```

```
#
```

```
=====
```

```
# 6. RUN BUFFER SIMULATIONS
```

```
#
```

```
=====
```

```
metrics_VOUT_MIN = Miller_evaluator_VOUT_min_err(**VOUT_min_params)
```

```
metrics_VOUT_MAX = Miller_evaluator_VOUT_max_err(**VOUT_max_params)
```

```
#
```

```
=====
```

```
# 7. ASSEMBLE FINAL SPECS DICTIONARY
```

```
#
```

```
=====
```

```
specs = {
```

```
    'Av': metrics_ac.get('AVD', 0.0),
```

```
    'BW': metrics_ac.get('BW', 0.0),
```

```
    'UGF': metrics_ac.get('UGF', 0.0),
```

```
    'CMRR': metrics_ac.get('CMRR', 0.0),
```

```
    'PM': PM,
```

```
    'Pw': metrics_dc.get('POWER', 1.0),
```

```
    'Slew_Rate': metrics_dc.get('SLEW_RATE', 0.0),
```

```
    'Av_VICM_MIN': metrics_CMIR_min.get('AVD', 0.0),
```

```
    'Av_VICM_MAX': metrics_CMIR_max.get('AVD', 0.0),
```

```
    'VOUT_MIN_Err': metrics_VOUT_MIN.get('VOUT_ERR', 1.0),
```

```
    'VOUT_MAX_Err': metrics_VOUT_MAX.get('VOUT_ERR', 1.0),
```

```
    'VOUT_CM': metrics_dc.get('VOUT_CM', 0.0)
```

```
}
```

```
return specs
```

```

def evaluate_5t_ota_ltspace(Wd, Ld, Wm, Lm, Wss, Lss, WB, Ib, CL, Vicm,
                             TGT_VICM_MIN, TGT_VICM_MAX, TGT_VOUT_MIN,
                             TGT_VOUT_MAX,
                             BASE_DIR='.'):

    LTSPICE_PATH =
r"C:\Users\Mohamed\AppData\Local\Programs\ADI\LTspice\LTspice.exe"
    """
    Executes NGSpice/LTspice testbenches for a 5T OTA and returns a specs
    dictionary.
    """

    # 1. Define Parameter Dictionaries
    parameters = {
        'Ld': Ld,
        'Wd': Wd,
        'Lm': Lm,
        'Wm': Wm,
        'LSS': Lss,
        'WSS': Wss,
        'WB': WB,
        'Ib': Ib,
        'vicm': Vicm,
        'CL': CL
    }

    CMIR_min_params = parameters.copy()
    CMIR_min_params['vicm'] = TGT_VICM_MIN

    CMIR_max_params = parameters.copy()
    CMIR_max_params['vicm'] = TGT_VICM_MAX

    VOUT_min_params = parameters.copy()
    VOUT_min_params['vicm'] = TGT_VOUT_MIN

    VOUT_max_params = parameters.copy()

```

```

VOUT_max_params['vicm'] = TGT_VOUT_MAX

# 2. Setup File Paths
full_path_AC = os.path.join(BASE_DIR, "OTA_5T_N_AC.sp")
full_path_DC = os.path.join(BASE_DIR, "OTA_5T_N_DC.sp")
full_path_BUFFER = os.path.join(BASE_DIR, "OTA_5T_N_buffer.sp")

# 3. Initialize Evaluators ---> FIX: Pass the ltspice_path
explicitly!
OTA_5T_evaluator_ac = SpiceEvaluator(full_path_AC,
ltspice_path=LTSPICE_PATH)
OTA_5T_evaluator_dc = SpiceEvaluator(full_path_DC,
ltspice_path=LTSPICE_PATH)
OTA_5T_evaluator_CMIR_min = SpiceEvaluator(full_path_AC,
ltspice_path=LTSPICE_PATH)
OTA_5T_evaluator_CMIR_max = SpiceEvaluator(full_path_AC,
ltspice_path=LTSPICE_PATH)
OTA_5T_evaluator_VOUT_min_err = SpiceEvaluator(full_path_BUFFER,
ltspice_path=LTSPICE_PATH)
OTA_5T_evaluator_VOUT_max_err = SpiceEvaluator(full_path_BUFFER,
ltspice_path=LTSPICE_PATH)

#
=====

# 4. RUN AC SIMULATIONS
#
=====

metrics_ac = OTA_5T_evaluator_ac(**parameters)

# FIX: Use absolute paths bound to BASE_DIR for the waveform reader
reader1 = SpiceWaveformReader(os.path.join(BASE_DIR,
"OTA_5T_N_AC.raw"))
freq, gain_db, phase_deg = reader1.get_ac_response("V(vout_dm)")

ugf_idx = np.argmin(np.abs(gain_db))
UGF = freq[ugf_idx]
PM = 180.0 - np.abs(phase_deg[ugf_idx] - phase_deg[0])

metrics_ac['UGF'] = float(np.real(UGF))

```

```

metrics_ac['AVD'] = float(np.max(gain_db))

# Get Common-Mode AC response
freq_cm, gain_db_cm, phase_deg_cm =
reader1.get_ac_response("V(vout_cm)")
metrics_ac['AVCM'] = float(gain_db_cm[0])
metrics_ac['CMRR'] = metrics_ac['AVD'] - metrics_ac['AVCM']

# CMIR Extremes
metrics_CMIR_min = OTA_5T_evaluator_CMIR_min(**CMIR_min_params)
reader_cmir_min = SpiceWaveformReader(os.path.join(BASE_DIR,
"OTA_5T_N_AC.raw"))
_, gain_db_min, _ = reader_cmir_min.get_ac_response("V(vout_dm)")
metrics_CMIR_min['AVD'] = float(np.max(gain_db_min))

metrics_CMIR_max = OTA_5T_evaluator_CMIR_max(**CMIR_max_params)
reader_cmir_max = SpiceWaveformReader(os.path.join(BASE_DIR,
"OTA_5T_N_AC.raw"))
_, gain_db_max, _ = reader_cmir_max.get_ac_response("V(vout_dm)")
metrics_CMIR_max['AVD'] = float(np.max(gain_db_max))

#
=====

# 5. RUN DC SIMULATION
#
=====

metrics_dc = OTA_5T_evaluator_dc(**parameters)

#
=====

# 6. RUN BUFFER SIMULATIONS
#
=====

metrics_VOUT_MIN = OTA_5T_evaluator_VOUT_min_err(**VOUT_min_params)
metrics_VOUT_MAX = OTA_5T_evaluator_VOUT_max_err(**VOUT_max_params)

#
=====

# 7. ASSEMBLE FINAL SPECS DICTIONARY

```

#

=====

```
specs = {
    'Av': metrics_ac.get('AVD', 0.0),
    'BW': metrics_ac.get('BW', 0.0),
    'UGF': metrics_ac.get('UGF', 0.0),
    'CMRR': metrics_ac.get('CMRR', 0.0),
    'PM': PM,
    'Pw': metrics_dc.get('POWER', 1.0),
    'Slew_Rate': metrics_dc.get('SLEW_RATE', 0.0),
    'Av_VICM_MIN': metrics_CMIR_min.get('AVD', 0.0),
    'Av_VICM_MAX': metrics_CMIR_max.get('AVD', 0.0),
    'VOUT_MIN_Err': metrics_VOUT_MIN.get('VOUT_ERR', 1.0),
    'VOUT_MAX_Err': metrics_VOUT_MAX.get('VOUT_ERR', 1.0),
    'VOUT_CM': metrics_dc.get('VOUT_CM', 0.0)
}

return specs
```